

Experiment - 1

Aim: Write a program to understand basic plots in python.

Theory:

Line Plot

A line plot displays data points on a graph and connects them with straight lines to show trends or changes over time. It is commonly used to visualize continuous data.

Bar Plot

A bar plot represents categorical data using rectangular bars.

Scatter Plot

A scatter plot shows individual data points plotted on a graph to illustrate the relationship or correlation between two variables.

Histogram

A histogram is used to display the distribution of numerical data by grouping values into intervals (bins). The height of each bar indicates the frequency of data within each interval.

Pie Chart

A pie chart is a circular diagram divided into sectors, where each sector represents a category's proportion of the total dataset, usually shown as percentages.

Box Plot (Box-and-Whisker Plot)

A box plot visually summarizes the distribution of a dataset using five key statistics: minimum, first quartile, median, third quartile, and maximum.

Code:

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 20, 15, 25, 30]
```

```
plt.figure()
```

```
plt.plot(x, y, marker='o')
```

```
plt.title("Line Plot")
```

```
plt.xlabel("X Values")
```

```
plt.ylabel("Y Values")
```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.figure()
```

```
plt.bar(x, y)
```

```
plt.title("Bar Plot")
```

```
plt.xlabel("Categories")
```

```
plt.ylabel("Values")
```

```
plt.show()
```

```
plt.figure()
```

```
plt.scatter(x, y)
```

```
plt.title("Scatter Plot")
```

```
plt.xlabel("X Values")
```

```
plt.ylabel("Y Values")
```

```
plt.show()
```

```
data = [1, 2, 2, 3, 3, 4, 4, 5, 6, 7, 8]
```

```
plt.figure()
```

```
plt.hist(data, bins=5, edgecolor='black')
```

```
plt.title("Histogram")
```

```
plt.xlabel("Value")
```

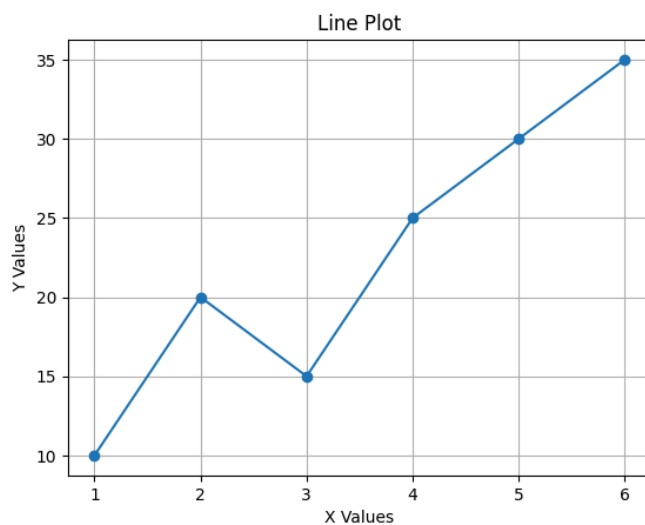
```
plt.ylabel("Frequency")
```

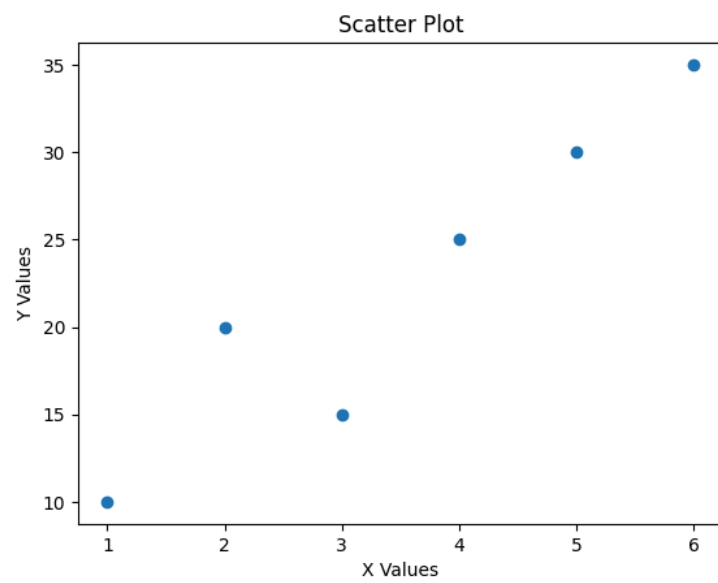
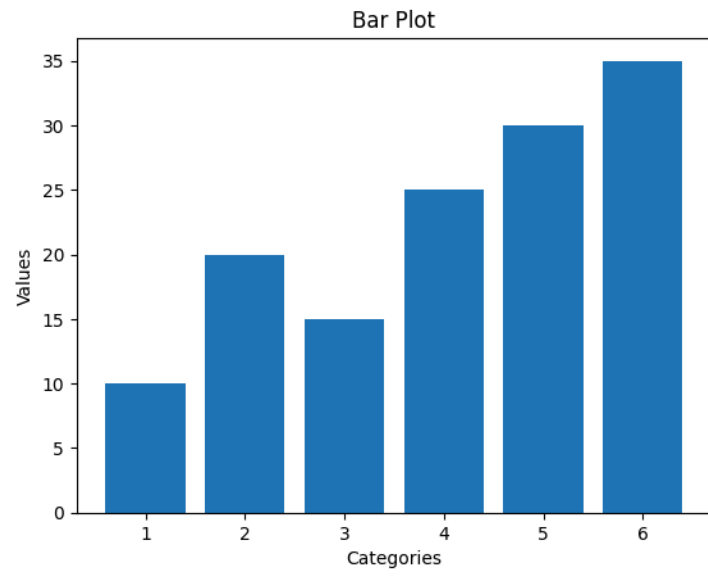
```
plt.show()
```

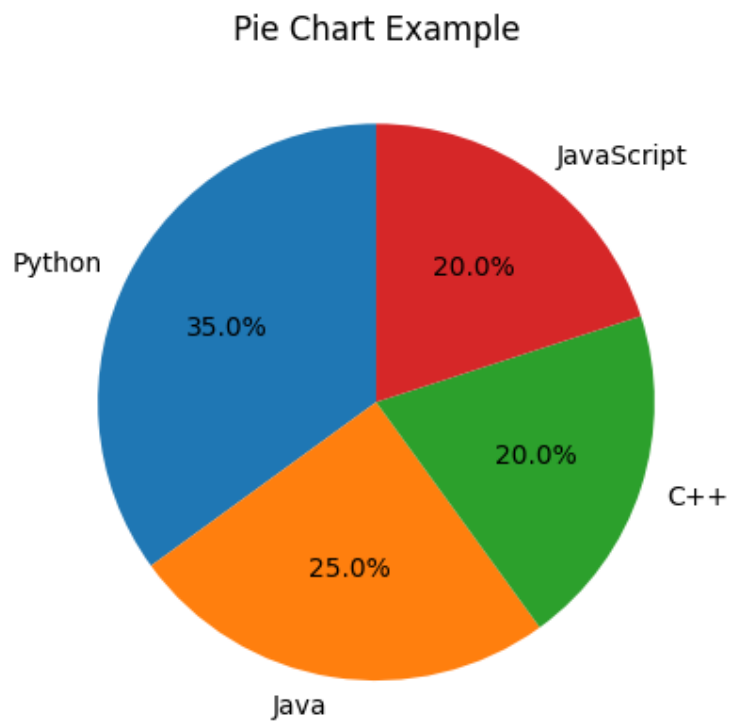
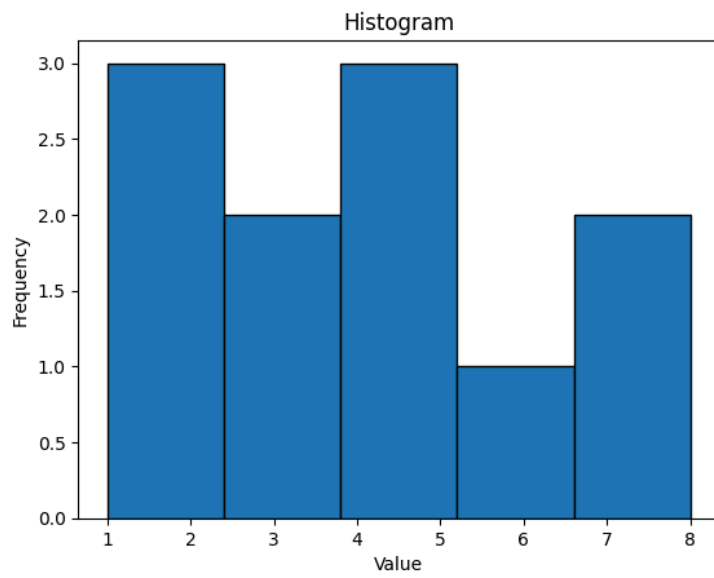
```
labels = ['Python', 'Java', 'C++', 'JavaScript']  
sizes = [35, 25, 20, 20]  
  
plt.figure()  
  
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)  
  
plt.title("Pie Chart Example")  
  
plt.show()
```

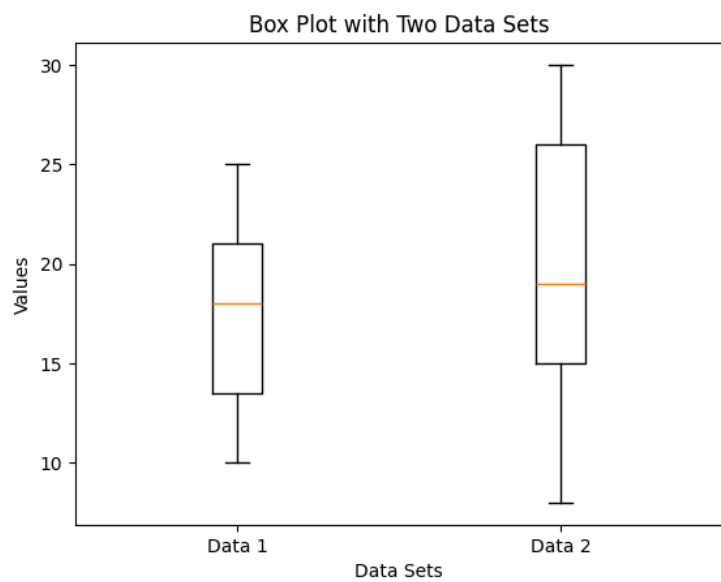
```
data1 = [10, 12, 15, 18, 20, 22, 25]  
data2 = [8, 14, 16, 19, 24, 28, 30]  
  
plt.boxplot([data1, data2])  
  
plt.title("Box Plot with Two Data Sets")  
  
plt.xlabel("Data Sets")  
  
plt.ylabel("Values")  
  
plt.xticks([1, 2], ['Data 1', 'Data 2'])  
  
plt.show()
```

Output:









Experiment - 2

Aim: To understand and perform basic matrix operations. (addition, subtraction, multiplication, transpose)

Theory:

1. Matrix Addition

Matrix addition is performed by adding corresponding elements of two matrices having the same dimensions.

2. Matrix Subtraction

Matrix subtraction is performed by subtracting corresponding elements of one matrix from another matrix of the same size.

3. Matrix Multiplication

Matrix multiplication is performed by multiplying rows of the first matrix with columns of the second matrix.

4. Matrix Transpose

The transpose of a matrix is obtained by converting rows into columns and columns into rows.

Code:

```
import numpy as np
```

```
A = np.array([[1, 2],  
              [3, 4]])
```

```
B = np.array([[5, 6],  
              [7, 8]])
```

```
print("Matrix A:\n", A)
```

```
print("Matrix B:\n", B)
```

Addition

C_add = A + B

print("\nA + B =\n", C_add)

Subtraction

C_sub = A - B

print("\nA - B =\n", C_sub)

Multiplication

C_mul = np.dot(A, B) # or A @ B

print("\nA * B =\n", C_mul)

Transpose

A_T = A.T

B_T = B.T

print("\nTranspose of A:\n", A_T)

print("\nTranspose of B:\n", B_T)

Output:


```
*** Matrix A:
    [[1 2]
     [3 4]]
Matrix B:
    [[5 6]
     [7 8]]

(Addition)
A + B =
    [[ 6  8]
     [10 12]]

(Subtraction)
A - B =
    [[-4 -4]
     [-4 -4]]

(Multiplication)
A * B =
    [[19 22]
     [43 50]]

Transpose of A:
    [[1 3]
     [2 4]]

Transpose of B:
    [[5 7]
     [6 8]]
```

Experiment - 3

Aim: Implementation of different Activation Functions to train neural network

Theory:

Activation functions are mathematical functions used in neural networks (NN) to introduce non-linearity into the model. They determine whether a neuron should be activated or not by transforming the input signal into an output signal.

Activation Functions:

Sigmoid Function – The sigmoid function transforms input values into a range between 0 & 1.

Tanh (Hyperbolic Tangent) – The tanh function outputs values between -1 and 1. Unlike the sigmoid function, it is zero-centered, which often helps the model learn more efficiently.

ReLU (Rectified Linear Unit) – ReLU outputs zero for negative input values and returns the input value itself for positive inputs.

Softmax Function – The softmax function converts a vector of values into probabilities that sum to 1. It is mainly used in the output layer for multi-class classification problems.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

# Fixed inputs, weights, bias and target

inputs = np.array([1, 0.5, -1, 2, 1.5])

weights = np.array([0.8, -0.4, 0.2, 1.0, -0.6])

bias = 0.3

target = 1

# Induced field calculation

v = np.dot(inputs, weights) + bias

print("Induced Field (v):", v)
```

```
# Activation outputs at induced field

sigmoid_out = 1 / (1 + np.exp(-v))

tanh_out = np.tanh(v)

relu_out = max(0, v)

leaky_relu_out = v if v > 0 else 0.01 * v

softmax_out = np.exp(v) / np.sum(np.exp(v))
```

```
print("\nActivation Outputs:")

print("Sigmoid:", sigmoid_out)

print("Tanh:", tanh_out)

print("ReLU:", relu_out)

print("Leaky ReLU:", leaky_relu_out)

print("Softmax:", softmax_out)
```

```
# Range of induced field for plotting
```

```
v_range = np.linspace(-10, 10, 400)
```

```
sigmoid = 1 / (1 + np.exp(-v_range))
```

```
tanh = np.tanh(v_range)
```

```
relu = np.maximum(0, v_range)
```

```
leaky_relu = np.where(v_range > 0, v_range, 0.01 * v_range)
```

```
exp_v = np.exp(v_range)
```

```
softmax = exp_v / np.sum(exp_v)
```

```
# Sigmoid plot

plt.figure()

plt.plot(v_range, sigmoid)

plt.axhline(0)

plt.axvline(0)

plt.title("Sigmoid Activation Function")

plt.xlabel("Induced Field (v)")

plt.ylabel("Output")

plt.grid(True)

plt.show()
```

```
# Tanh plot

plt.figure()

plt.plot(v_range, tanh)

plt.axhline(0)

plt.axvline(0)

plt.title("Tanh Activation Function")

plt.xlabel("Induced Field (v)")

plt.ylabel("Output")

plt.grid(True)

plt.show()
```

```
# ReLU plot

plt.figure()

plt.plot(v_range, relu)

plt.axhline(0)
```

```
plt.axvline(0)

plt.title("ReLU Activation Function")

plt.xlabel("Induced Field (v)")

plt.ylabel("Output")

plt.grid(True)

plt.show()
```

```
# Leaky ReLU plot

plt.figure()

plt.plot(v_range, leaky_relu)

plt.axhline(0)

plt.axvline(0)

plt.title("Leaky ReLU Activation Function")

plt.xlabel("Induced Field (v)")

plt.ylabel("Output")

plt.grid(True)

plt.show()
```

```
# Softmax plot

plt.figure()

plt.plot(v_range, softmax)

plt.axhline(0)

plt.axvline(0)

plt.title("Softmax Activation Function")

plt.xlabel("Induced Field (v)")

plt.ylabel("Probability")
```

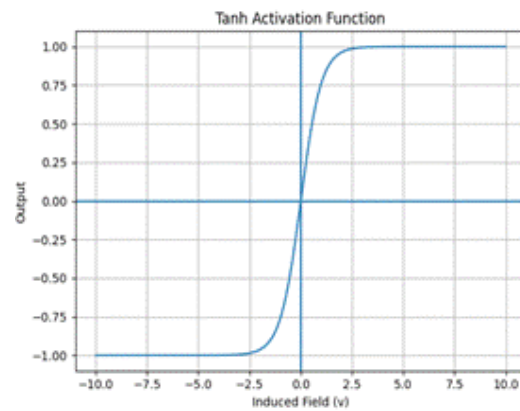
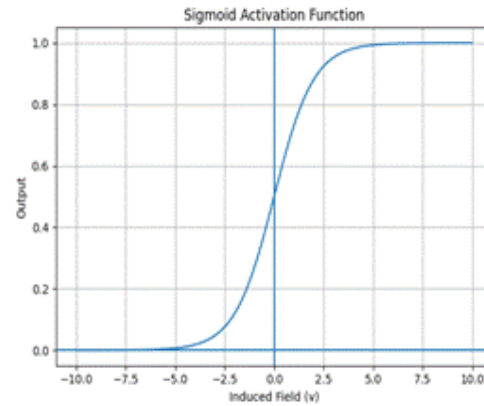
```
plt.grid(True)
```

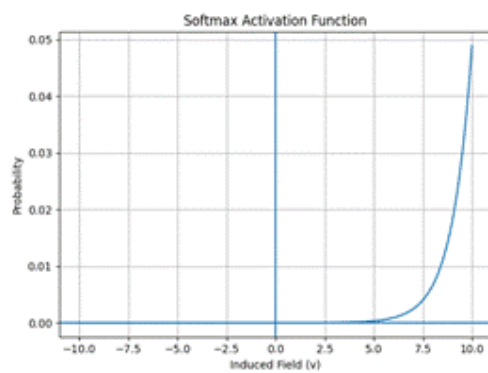
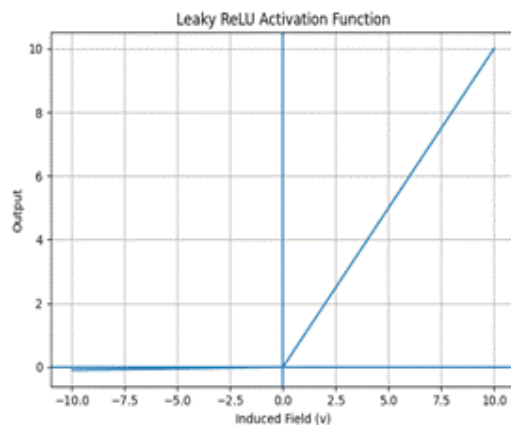
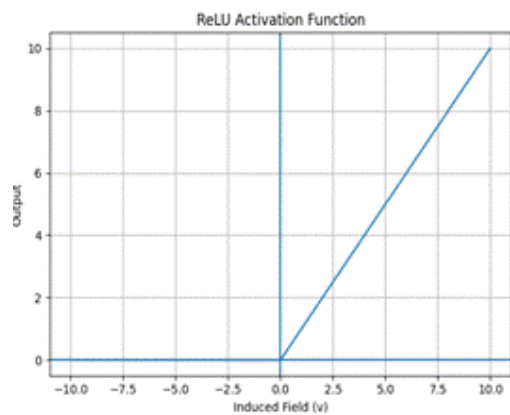
```
plt.show()
```

Output:

```
*** Induced Field (v): 1.8

Activation Outputs:
Sigmoid: 0.8581489350995123
Tanh: 0.9468060128462683
ReLU: 1.8
Leaky ReLU: 1.8
Softmax: 1.0
```





Experiment - 4

Aim: To understand weight and bias effect on output (Delta learning rule).

Theory:

The Delta Learning Rule (also called the Widrow–Hoff Rule) is a supervised learning algorithm used to minimize error in single-layer neural networks such as Adaline (Adaptive Linear Neuron).

Basic Neuron Model

A single neuron computes:

$$y=wx+b$$

Where:

- $x \rightarrow$ input
- $w \rightarrow$ weight
- $b \rightarrow$ bias
- $y \rightarrow$ predicted output

Error Definition

The learning goal is to reduce the error between:

- **Target output (t)**
- **Predicted output (y)**

Error:

$$e=t-y$$

Code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
X = np.array(list(map(float, input("Enter input values (space separated): ").split())))
```

```
T = np.array(list(map(float, input("Enter target values (space separated): ").split())))
```



```

# Check length

if len(X) != len(T):

    print("Error: Input and Target arrays must be same length!")

    exit()


w = float(input("Enter initial weight: "))

b = float(input("Enter initial bias: "))

learning_rate = float(input("Enter learning rate (e.g., 0.01): "))

epochs = int(input("Enter number of epochs: "))


print("\nTraining Started...\n")


# ----- TRAINING -----

for epoch in range(epochs):

    total_error = 0


    for x, t in zip(X, T):

        y = w * x + b

        error = t - y

        total_error += error**2


    # Delta rule update

    w = w + learning_rate * error * x

    b = b + learning_rate * error

```

```

print(f"Epoch {epoch+1}: Weight={w:.4f}, Bias={b:.4f}, Error={total_error:.4f}")

print("\nFinal Weight:", w)

print("Final Bias:", b)

```

Output:

```

*** Enter input values (space separated): 3 4 5 6
Enter target values (space separated): 4 5 6 7
Enter initial weight: 0.1
Enter initial bias: 0.1
Enter learning rate (e.g., 0.01): 0.05
Enter number of epochs: 15

Training Started...

Epoch 1: Weight=1.0718, Bias=0.3901, Error=17.7335
Epoch 2: Weight=1.0896, Bias=0.4029, Error=0.1830
Epoch 3: Weight=1.0888, Bias=0.4102, Error=0.1326
Epoch 4: Weight=1.0878, Bias=0.4174, Error=0.1287
Epoch 5: Weight=1.0867, Bias=0.4244, Error=0.1255
Epoch 6: Weight=1.0856, Bias=0.4314, Error=0.1225
Epoch 7: Weight=1.0846, Bias=0.4383, Error=0.1196
Epoch 8: Weight=1.0836, Bias=0.4451, Error=0.1167
Epoch 9: Weight=1.0826, Bias=0.4519, Error=0.1139
Epoch 10: Weight=1.0816, Bias=0.4585, Error=0.1111
Epoch 11: Weight=1.0806, Bias=0.4651, Error=0.1084
Epoch 12: Weight=1.0796, Bias=0.4716, Error=0.1058
Epoch 13: Weight=1.0786, Bias=0.4780, Error=0.1033
Epoch 14: Weight=1.0777, Bias=0.4843, Error=0.1008
Epoch 15: Weight=1.0767, Bias=0.4906, Error=0.0984

Final Weight: 1.0767325812708326
Final Bias: 0.4905582798068541

```

Experiment - 5

Aim: To optimize ANN using the Gradient Descent method.

Theory:

Artificial Neural Networks (ANNs) are computational models made up of interconnected neurons that learn patterns from data. The performance of an ANN depends on how well its weights and biases are adjusted during training.

Gradient Descent is an optimization algorithm used to minimize the loss function, which represents the error between predicted and actual outputs. It works by iteratively updating the weights in the direction of the negative gradient of the loss function.

During training, forward propagation computes the output, and backpropagation calculates the gradients of the loss with respect to each weight. The weights are then updated using the rule.

$$W = W - \eta (\partial L / \partial W)$$

Thus, Gradient Descent helps the ANN learn optimal parameters and improve accuracy by reducing the error over multiple iterations.

Code:

```
import numpy as np
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):
```

```
    return x * (1 - x)
```

```
X = np.array([[0,0],[0,1],[1,0],[1,1]])
```

```
y = np.array([[0],[1],[1],[0]])
```

```
input_neurons = 2
```

```
hidden_neurons = 2
```

```
output_neurons = 1
```

```
np.random.seed(42)
```

```
wh = np.random.uniform(size=(input_neurons, hidden_neurons))
```

```
bh = np.random.uniform(size=(1, hidden_neurons))
```

```
wo = np.random.uniform(size=(hidden_neurons, output_neurons))
```

```
bo = np.random.uniform(size=(1, output_neurons))
```

```
learning_rate = 0.1
```

```
epochs = 10000
```

```
print("\n--- Training ANN using Gradient Descent ---\n")
```

```
for epoch in range(epochs):
```

```
    hidden_layer_input = np.dot(X, wh) + bh
```

```
    hidden_layer_output = sigmoid(hidden_layer_input)
```

```
    output_layer_input = np.dot(hidden_layer_output, wo) + bo
```

```

predicted_output = sigmoid(output_layer_input)

error = y - predicted_output
d_predicted_output = error * sigmoid_derivative(predicted_output)

error_hidden_layer = np.dot(d_predicted_output, wo.T)
d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_layer_output)

wo += np.dot(hidden_layer_output.T, d_predicted_output) * learning_rate
bo += np.sum(d_predicted_output, axis=0, keepdims=True) * learning_rate
wh += np.dot(X.T, d_hidden_layer) * learning_rate
bh += np.sum(d_hidden_layer, axis=0, keepdims=True) * learning_rate

if epoch % 1000 == 0:
    loss = np.mean(np.square(y - predicted_output))
    print(f"Epoch {epoch}, Loss: {loss:.4f}")

print("\n--- Training Complete ---\n")

print("Final Predicted Output (after training):\n", predicted_output.round(2))
print("Target Output:\n", y)

```

Output:

```
***
--- Training ANN using Gradient Descent ---

Epoch 0, Loss: 0.3247
Epoch 1000, Loss: 0.2406
Epoch 2000, Loss: 0.1960
Epoch 3000, Loss: 0.1207
Epoch 4000, Loss: 0.0305
Epoch 5000, Loss: 0.0125
Epoch 6000, Loss: 0.0074
Epoch 7000, Loss: 0.0051
Epoch 8000, Loss: 0.0038
Epoch 9000, Loss: 0.0031

--- Training Complete ---

Final Predicted Output (after training):
[[0.05]
 [0.95]
 [0.95]
 [0.05]]
Target Output:
[[0]
 [1]
 [1]
 [0]]
```

